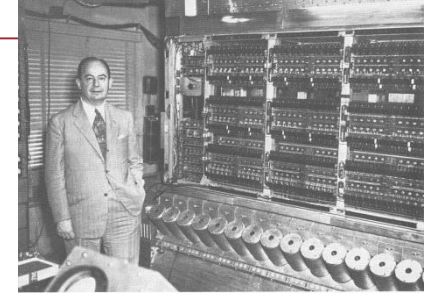


Betriebssysteme

Kapitel 7: Programmierung und Architektur

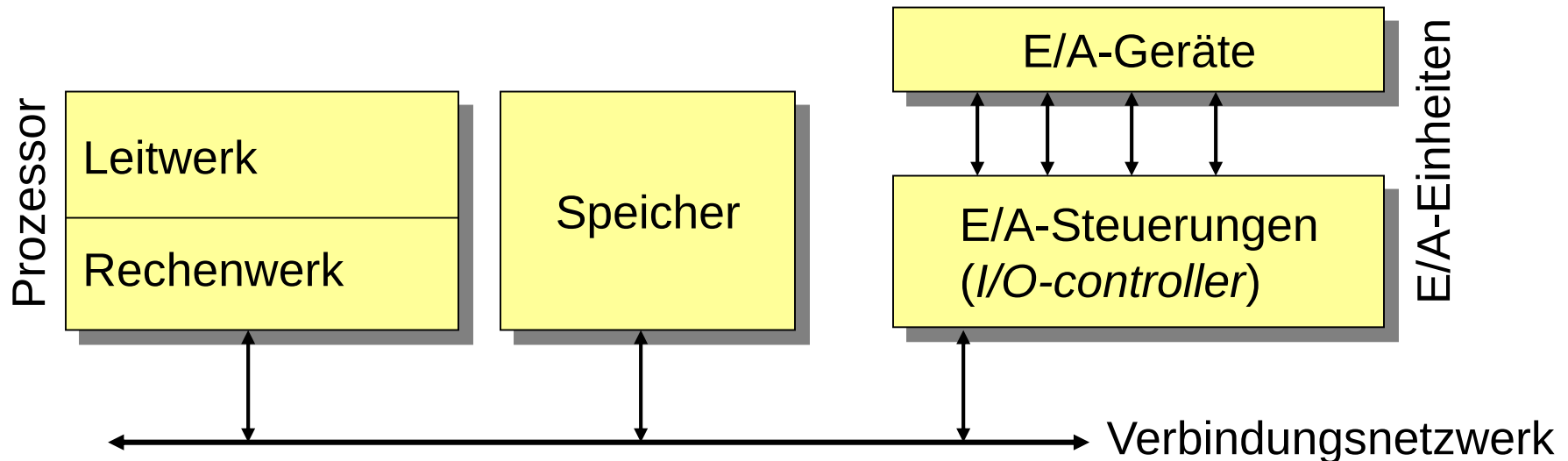
Prof. Dr. Raphael Herding

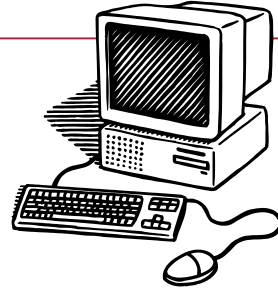


Das Von Neumann-Modell (1)

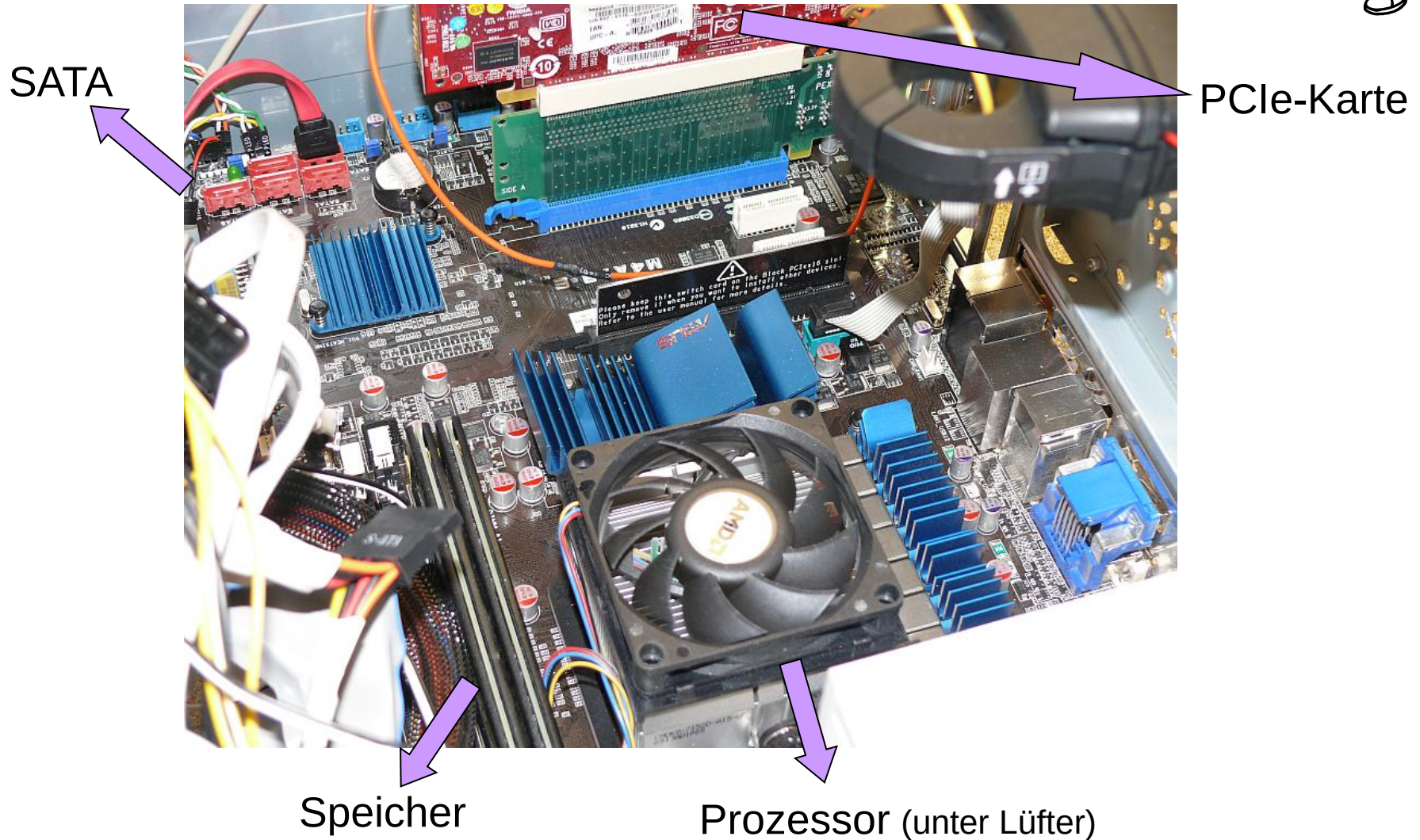
Fast alle der heute üblichen Rechner gehen auf den Von Neumann-Rechner mit folgenden Eigenschaften zurück:

1. Die Rechanlage besteht aus den Funktionseinheiten **Speicher**, **Leitwerk** (bzw. Steuerwerk, engl. *controller*), dem **Rechenwerk** (engl. *data path*) und **Ein-/Ausgabe-Einheiten**.





Wo sind diese Komponenten auf PC-Boards?



Das Von Neumann-Modell (2)

2. Die Struktur der Anlage ist unabhängig vom bearbeiteten Problem. Die Anlage ist **speicherprogrammierbar**.
3. **Anweisungen und Operanden** (einschl. Zwischenergebnissen) werden **in demselben physikalischen Speicher** gespeichert.
4. Der Speicher wird in **Zellen gleicher Größe** geteilt. Die Zellnummern heißen **Adressen**.
5. Das Programm besteht aus einer Folge von elementaren **Befehlen**, die **in der Reihenfolge der Speicherung bearbeitet** werden.
6. Abweichungen von der Reihenfolge sind mit (bedingten oder unbedingten) **Sprungbefehlen** möglich.

Das Von Neumann-Modell (3)

7. Es werden **Folgen von Binärzeichen** (nachfolgend Bitvektoren genannt) verwendet, um alle Größen darzustellen.
8. Die Bitvektoren erlauben **keine explizite Angabe des repräsentierten Typs**. Aus dem Kontext heraus muss stets klar sein, wie die Bitvektoren zu interpretieren sind.

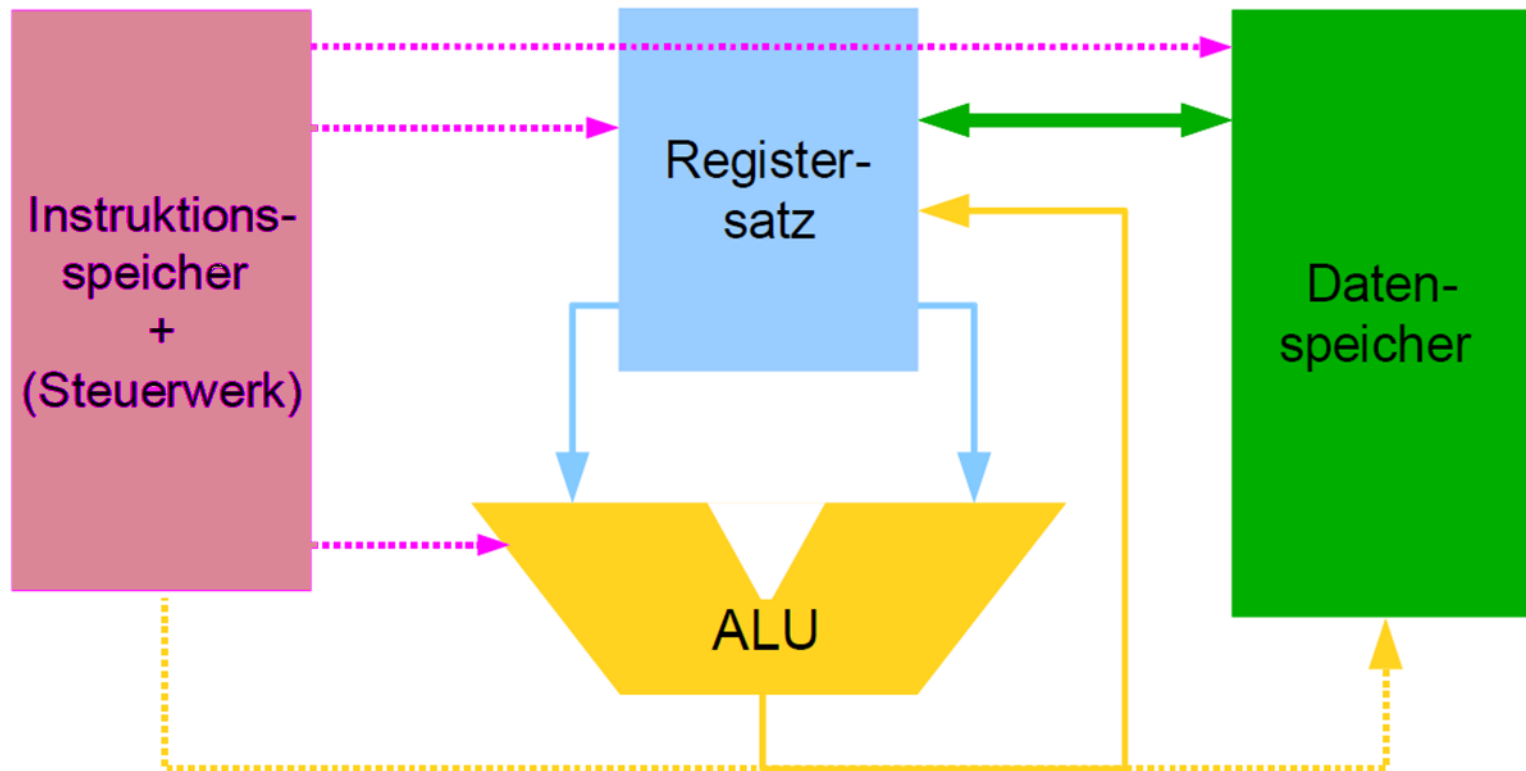
Alternative:

Typ	Wert
-----	------

MIPS Architekturskizze

Vereinfachte Sicht einer MIPS

- Informationsflüsse
 - Steuersignale, Speicherdaten, Registerdaten, Adressen, Instruktionen





Der MIPS Befehlssatz

Beispiel: MIPS (~ *Machine with no Interlocked Pipe Stages*)

≠ MIPS (*million instructions per second*)

Entwurf Anfang der 80er Jahre

Warum MIPS?

- Weitgehend sauberer und klarer Befehlssatz
- Kein historischer Ballast
- Basis der richtungsweisenden Bücher von Hennessy/Patterson
- Simulator verfügbar
- MIPS außerhalb von PCs weit verbreitet
(bei Druckern, Routern, Handys)

Begriffe

- **MIPS-Befehle** sind elementare Anweisungen an die MIPS-Maschine
- Ein **MIPS-Maschinenprogramm** ist eine konkrete Folge von MIPS-Befehlen
- Die **MIPS-Maschinensprache** ist die Menge möglicher MIPS-Maschinenprogramme

- Entsprechendes gilt für die Assemblerprogramm-Ebene
- Vielfach keine Unterscheidung zwischen beiden Ebenen

MIPS Instruktionsformate

Generell

- 32-Bit Wörter
- 32 Operandenregister
- 6 Bit für Operationscode
- je 5 Bit Registerindex für Operand 1, Operand 2 und Resultat
- 5 Bit Verschiebung (*shift amount*)
- 6 Bit Funktionscode

Instruktionsformate

- R-Format für Register/ALU-Operationen
- I-Format für *Immediate*-Operanden
- J-Format für *Jump*-Befehle

Opcode(6)	rs(5)	rt(5)	rd(5)	shamt(5)	funct(6)
Opcode(6)	rs(5)	rt(5)	I-Konstante(16)		
Opcode(6)	Sprungziel(26)				

Binäres Maschinenprogramm

Instruktionen im Programmspeicher

- Instruktionen und Programmausführung ab Adresse 0x00400000

0x00400000: 00000000 00011111 00001000 00100000

0x00400004: 00000011 11100000 00001000 00100010

0x00400008: 00000000 00000001 11111000 00101010

0x0040000C: 00100000 00000111 10101111 11111110

0x00400010: 10101100 01100111 10101111 11110000

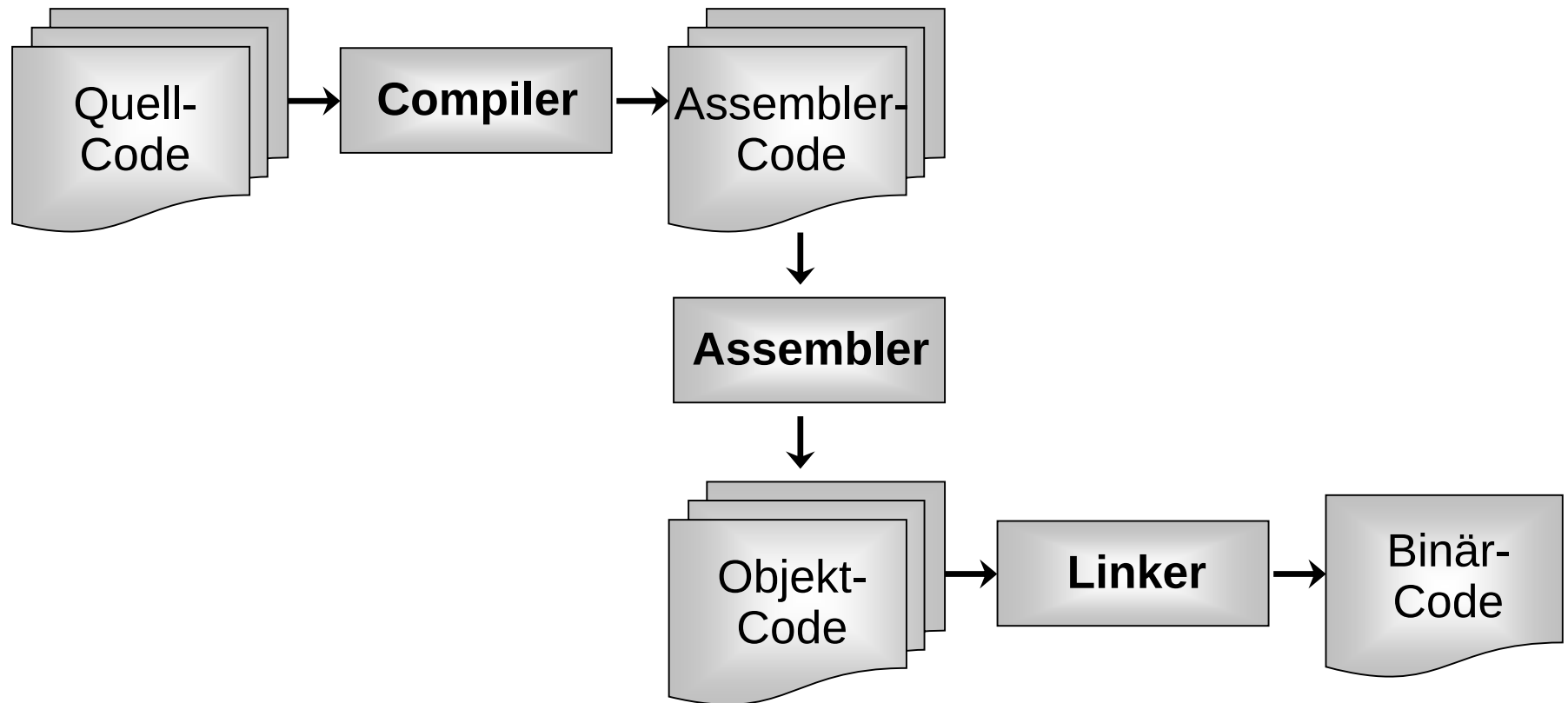
0x00400014: 00001000 00010000 00000000 00000000

Legende

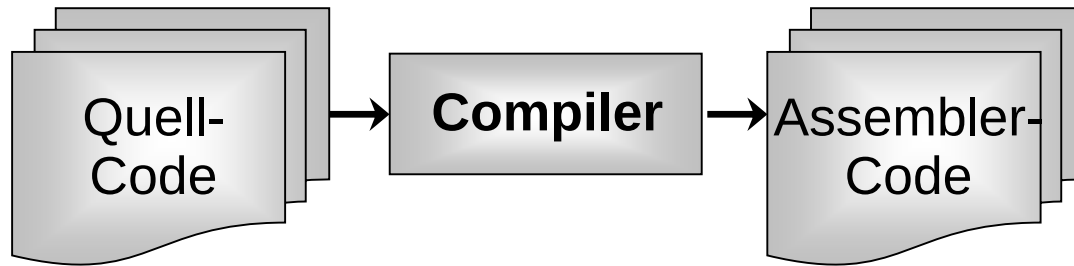
- | | | |
|--------------------|--------------------|---------------------|
| – Hex-Adresse | – rt-Registerindex | – Funktionscode |
| – Opcode | – rd-Registerindex | – Immediate Operand |
| – rs-Registerindex | – shift amount | – Sprungziel |

- ☞ Programmerstellung direkt als Binärcode oder Hexcode fehleranfällig
Nur im äußersten Notfall in Betracht zu ziehen → Assemblersprache

Werkzeuge zur Code-Generierung



- Warum ist Java hardwareunabhängig?
- Warum ist c++ nicht hardwareunabhängig?



Quellcode

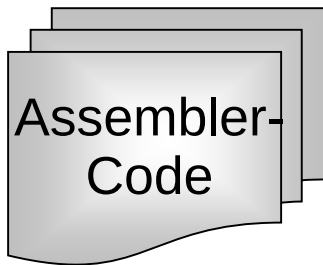
- Von Menschen les- / verstehbare Programmiersprache
- Hochsprachliche Konstrukte: Klassen, Prozeduren, Schleifen, Variablen
- Hohes Abstraktionsniveau: Maschinenunabhängige Algorithmen

Assemblercode

- Symbolischer Maschinencode
- Für Menschen eingeschränkt les- / verstehbar
- Maschinensprachen-Konstrukte: ALU-Befehle, Register, ...
- Niedriges Abstraktionsniveau: Maschinenabhängige Darstellung

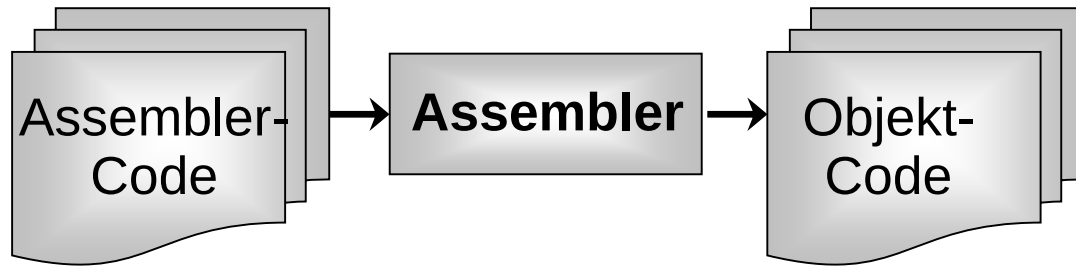
Compiler

- Übersetzt Quell- in Assemblercode
- Optimiert Code (z.B. bzgl. Laufzeit, Codegröße, Energieeffizienz, ...)



- Lesbare Textdarstellung
- Keine / wenige reale Adressen
- Statt dessen: Symbolische Adressen
z.B. **init**, **loop**, **arr**

<code>.data</code>	
<code>arr: .word 0:10</code>	<i># Reserviere 10 Array-Elemente</i>
<code>.text</code>	
<code>init: la \$7, arr(\$0)</code>	<i># Lade Adresse von array arr nach \$7</i>
<code>addi \$8, \$7, 40</code>	<i># Setze End-Index</i>
<code>addi \$9, \$0, 0x41</code>	<i># Lade 'A' nach \$9</i>
<code>loop: sw \$9, 0(\$7)</code>	<i># Speichere 'A' in arr</i>
<code>addi \$7, \$7, 4</code>	<i># Setze Index auf nächstes array-Element</i>
<code>beq \$7, \$8, end</code>	<i># Schleifen-Abbruch?</i>
<code>j loop</code>	<i># Schleifen-Rücksprung</i>
<code>end: addi \$2, \$0, 10</code>	<i># Systemaufruf zum Beenden</i>
<code>syscall</code>	

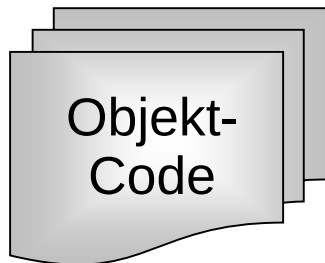


Objektcode

- Binärdarstellung von Assemblercode, nicht mehr lesbar
- Keine Klartext-Mnemonics, statt dessen 0/1-Sequenzen
- Wenn möglich, symbolische Adressen durch reale ersetzt

Assembler

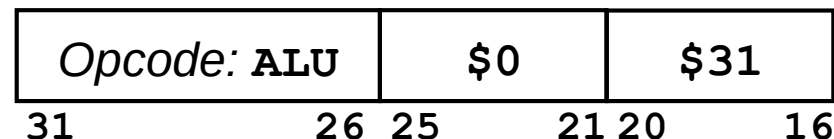
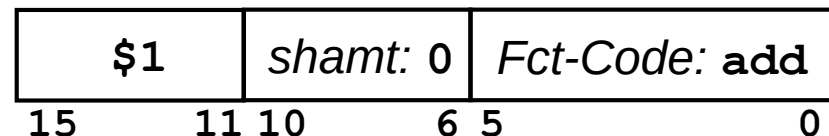
- Zeilenweise Übersetzung
Assembler-Befehle → Maschinen-Befehle
- Innerhalb eines Assembler-Files: Adress-Auflösung



Übersetzung

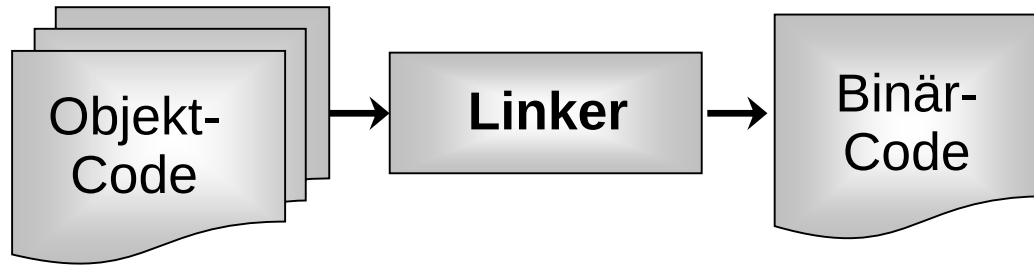
`add $1,$0,$31`

000000 00000 11111 00001 00000 100000



Adress-Auflösung

- Symbolische Adresse **arr** in gleichem Assembler-File deklariert:
 - Symbol **arr** ist Assembler bekannt
 - Ersetzung von **arr** durch relative Adresse, relativ in dem Objekt-File
- **arr** ist Assembler unbekannt:
 - Adress-Auflösung erfolgt später



Binärcode

- Ausführbare Programm-Darstellung
- Alle symbolischen Adressen durch reale ersetzt
- Niedrigstes Abstraktionsniveau

Linker

- Vereinigung vieler Objektcodes und Bibliotheken zu einem ausführbaren Programm
- Symbol-Auflösung mit Hilfe von Objektcode-Bibliotheken
- Code-Anordnung im Speicher

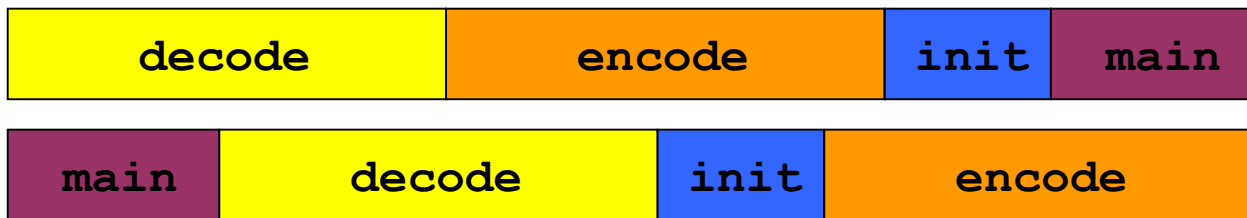
Binär-
Code

Beispiel Symbol-Auflösung

- Objektcode enthält Aufruf einer externen Funktion: `jal init`
- Suche `init` in allen anderen Objektcodes & Bibliotheken
- Füge Code von `init` dem Binärcode zu

Beispiel Speicher-Layout des Binärcodes

- Binärcode besteht aus Funktionen `init`, `decode`, `encode`, `main`



- Speicher-Anordnung definiert abschließend reale Adressen

Bestandteile eines Binär-Programms

- Header: Identifikator (*Magic Number*), Längenangaben, Datum, ...
- Codesegment / Textsegment: Maschinenbefehle des Programms, Konstanten, *Strings*, Sprungtabellen, ...
- Datensegment:
 - (initialisierte) Variablen
 - nichtinitialisierte Variablen (*Block Storage Segment, BSS*)
- Symboltabelle: *Labels* im-/exportierter Variablen & Einsprungpunkte, ...
- *Debugging*-Information:
Zeilennummern, Variablen, Datenstrukturen & Prozedurnamen
- Relokationsinformationen
 - Adressen, welche bei einer Verschiebung des Programms angepasst werden müssen

Dateiformate für Binär-Programme

a.out

- Klassisches UNIX-Format
- Details sind plattformabhängig

COFF (*Common Object File Format*)

- Idee des einheitlichen Formats über alle Plattformen
- Headervariable spezifiziert Prozessorplattform
- Variante PE-COFF (*Portable Executable ...*): Windows-Welt

exe (*Executable*)

- Windows

Was fehlt noch?

Der Lader (*Loader*)

- Laden eines Binär-Programms an die gewünschte Stelle im Hauptspeicher
- „Relozierung“ bedeutet, Programme im Hauptspeicher zu verschieben und Referenzen auf Speicheradressen anzupassen
- Verwaltung und Berücksichtigung von Relokationsinformationen

(Höhere) Programmiersprachen im Vergleich (1)

Wesentliche Erleichterung für die Programmierarbeit

- Quelltext zum Beispiel in Java/C++/C
- Typenkonzept für Zuweisungen und Funktionsaufrufe
- Mehr als ein Maschinenbefehl pro Statement
- Compiler verwendet u.U. nicht alle Befehle
- Geschwindigkeitsnachteil
- Bessere Produktivität

```
...
max = 40;
val = 'A';
do {
    arr[inx++] = val;
}
while ( inx < max );
...
```

Entsprechendes Programm in Assembler

- Auf die MIPS-CPU bezogene Befehlskürzel
- Gute Auslastung der vorhandenen Register
- Pro Zeile ein Maschinenbefehl
- Kein Typenkonzept

```
init:  la    $7, arr($0)
        addi $8, $7, 40
        addi $9, $0, 'A'
loop:  sw    $9, 0($7)
        addi $7, $7, 4
        beq  $7, $8, end
        j    loop
```